Introduction to Data Science
R Tutorial
Week 1

Ilse Cuevas Andrade and Dr. Gah-Yi Ban
Imperial College Business School

Introduction to programming in R Notebook

This tutorial includes more exercises about R variables, data types and built-in functions.

An important note to make about variables is *naming*. Variable names must *start with a letter* and should *only* contain:

- letters
- numbers
- underscores (to separate words)

When naming variables, consider readability. Names like `x`, `y` and `z` are not very descriptive and do not make for very readable code. Great variable names will give information about the context and maybe even units!

Consider the difference between the following variable names:

- `GDP` versus `uk_GDP_2018`
- `GDP_growth` versus `uk_GDP_growth_rate_2019`

```
# GDP unit : m (million)
uk_GDP_2018 <- 2203005
uk_GDP_2019 <- 2238348
uk_GDP_2020 <- 1991439
uk_GDP_2021 <- 2142738
uk_GDP_2022 <- 2230625
```

Run the chunk above then use the variables to find the annual GDP (Gross Domestic Product) growth rate in the UK between 2018 and 2022. To calculate the annual GDP growth rate based on the provided GDP, use the following equation:

$$GDP_{growth} = (GDP_{current} - GDP_{previous})/GDP_{previous} * 100$$

```
# Use the above variables to complete the equation below
uk_GDP_growth_rate_2019 <- (uk_GDP_2019 - uk_GDP_2018)/uk_GDP_2018 * 100

print(uk_GDP_growth_rate_2019)
```

```
## [1] 1.604309
```

```
uk_GDP_growth_rate_2020 <- (uk_GDP_2020 - uk_GDP_2019)/uk_GDP_2019 * 100

print(uk_GDP_growth_rate_2020)
```

```
## [1] -11.03086
```

```r
uk_GDP_growth_rate_2021 <- (uk_GDP_2021 - uk_GDP_2020)/uk_GDP_2020 * 100

print(uk_GDP_growth_rate_2021)
```

```
## [1] 7.597471
```

Numeric and character variables are two of the most common types in R, but as you will see over the course of the module, there are plenty of other types too.

Dates

Because we use and rely on dates so often for data analyses, R has a special data type just for dates, called *Date*. Using the Date type instead of a numeric or character type allows for proper comparison and calculation of values that represent days, months and years.

We will often find data where dates are given to us as a character type which need to be converted to the Date type before we can start working with them.

To convert, we use the `as.Date()` function and pass in the character date and the format it is in (order of year, month and day). Use the following special notation to specify the format of the date:

- %Y for the 4-digit year
- %m for the 2-digit month
- %d for the 2-digit day

```r
# Messy date data as characters (Dec 7 2022)
good_day <- "20221207"

# Convert with as.Date
as.Date(good_day, format = "%Y%m%d")
```

```
## [1] "2022-12-07"
```

```r
# Sometimes our date has separators (Aug 18 2023)
great_day <- "18-08-2023"

# as.Date() can handle that too
as.Date(great_day, format = "%d-%m-%Y")
```

```
## [1] "2023-08-18"
```

Exercise 2: Dates

Convert the following dates by filling in values for the `format` argument:

```r
# July 1 1867
as.Date("07/01/1867", format = "%m/%d/%Y")
```

```
## [1] "1867-07-01"
```

```r
# May 22 1927
as.Date("22-05 1927", format = "%d-%m %Y")
```

```
## [1] "1927-05-22"
```

```r
# September 5 2022
as.Date("2022 09 05", format = "%Y %m %d")
```

```
## [1] "2022-09-05"
```

```r
# January 1 1970 (hint: use %y to specify a 2-digit year)
as.Date("70-01-01", format = "%y-%m-%d")
```

```
## [1] "1970-01-01"
```

```r
# December 31 1999 (hint: use %b to specify an abbreviated month name)
as.Date("31 Dec 1999", format = "%d %b %Y")
```

```
## [1] "1999-12-31"
```

Built-in Functions

`as.Date` used earlier is a kind of a *built-in function* in R. For many other mathematical operations in R, we can also use built-in functions in R. Functions are commonplace in most programming languages, and we can think of them as bundling some common functionality into an easy-to-reuse command.

For example, let's use the `round()` function to practice rounding numbers.

```r
Pi <- 3.1415926

# Rounding Pi to the nearest whole number
round(Pi)
```

```
## [1] 3
```

Often (but not always) functions will operate on some input. We *pass* this information to the function as an *argument* (A.K.A parameter), entered inside round brackets.

When a function needs additional information to know how to behave, we can pass in additional arguments.

```r
# Rounding Pi to two decimal places
round(Pi, digits = 2)
```

```
## [1] 3.14
```

Arguments are separated by commas, and the order in which they are passed is important (also note whitespace). Where possible, refer to arguments by name.

A popular functionality across most programming languages is the ability to print text or numbers to the screen. In R, this is achieved with `print()`

```r
print(Pi)
```

```
## [1] 3.141593
```

```r
print("Hello World!")
```

```
## [1] "Hello World!"
```

Notice how an argument to the `print()` function is enclosed in double-quotes, as is its respective output. Quotation marks indicate non-numeric characters.

The `print()` function will accept either *numeric* types or *character* types as arguments, but not all functions are so flexible.

Google can help with finding the right function for the job. Once you know the function name, we can ask RStudio to show us detailed documentation.

Run the code chunk below. You won't see any output in the notebook, but a help page should open in the bottom right panel of RStudio.

```r
?print
```

```
## starting httpd help server ... done
```

HINT It may be easier to ask for help in the Console (the bottom-left pane in RStudio)

Exercise 3: Exploring other functions

Use help to learn about these two functions:

- `sqrt()`
- `floor()`

Then use the empty code chunk below to try the functions. Pass in arguments and check the output! I encourage you to practice commenting your code.

```r
a <- 4
b <- 3.8
sqrt(a) # Find the square root of a
```

```
## [1] 2
```

```r
floor(b) # Round down to nearest integer of b
```

```
## [1] 3
```

It will become very useful to know what type of data your variables hold. For this, we can use the `class()` function. Use the `class()` function to determine the type (or class) of the three variables used above - `x` , `y` , `z`

```r
x <- 4
y <- 4.5
z <- "Imperial"

class(x)
```

```
## [1] "numeric"
```

```r
class(y)
```

```
## [1] "numeric"
```

```r
class(z)
```

```
## [1] "character"
```

```r
z<- TRUE
class(z)
```

```
## [1] "logical"
```

The End